CS536 Final Project

Author

1 Motivation

Throughout this course, we have mostly considered learning settings where there is a single entity handling training and testing. However, these techniques are not always applicable to scenarios like edge computing and IoT devices. Often, there may be multiple devices with disjoint data sets that need to be aggregated to train a shared model. These devices may also have bandwidth or privacy requirements, preventing users from directly aggregating data then training. This is addressed by techniques in federated learning [4].

There are several ways data can be split among multiple devices. In the first case, each device can hold a subset of the data, recording different events with identical features. This is the problem that horizontal federated learning solves [2]. Alternatively, perhaps each device records some information about the same event, but they hold a disjoint set of features. We address this problem with vertical federated learning [1].

2 Vertical Federated Learning

In a general learning problem, we have $\mathbf{x} \in \mathbb{R}^m$ as an input vector and we want to learn some function $f : \mathbb{R}^m \to \mathbb{R}$ and predict the value $y = f(\mathbf{x})$. Consider the k party case, where we have $m = \sum_{i=1}^{k} m_i$, and $\mathbf{x} = \bigoplus_{i=1}^{m} \mathbf{x}_i \in \bigoplus_{i=1}^{m} \mathbb{R}^{m_i}$. Party i can only see $\mathbf{x}_i, y$, and needs to cooperate with other parties to predict $f(\mathbf{x}_1 \oplus \mathbf{x}_2)$ without divulging their own data. We implement a simplified version of [3] for reasons we discuss later.

For simplicity, consider the case of linear regression. Take $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} = \sum_{i=1}^{k} \mathbf{x}_i \cdot \mathbf{w}_i$ for $\mathbf{w} \in \mathbb{R}^m$, where $\mathbf{w} = \sum_{i=1}^{k} \mathbf{w}_i \in \bigoplus_{i=1}^{m} \mathbb{R}^{m_i}$ and party i only sees $\mathbf{w}_i$. Given some $\mathbf{x}_i$, y, party i will compute and broadcast $\mathbf{x}_i \cdot \mathbf{w}_i$, then collect $\hat{y} = \sum_{i=1}^{k} \mathbf{x}_i \cdot \mathbf{w}_i$ from other parties. The goal is to minimize the squared error $\mathcal{L} = (y - \hat{y})^2$.

To train this model, some data $\mathbf{X} = (\mathbf{x}^1, \ldots, \mathbf{x}^N)^\intercal \in \mathbb{R}^{N \times m}$ and output $\mathbf{y}(y_1, \ldots, y_N)^\intercal \in \mathbb{R}^N$ is partitioned to $\mathbf{X}_i \in \mathbb{R}^{N \times m_1}$. Party i obtains $\mathbf{X}_i$, $\mathbf{y}$, and initializes $\mathbf{w}_i$ to a random vector. Then we run several rounds of gradient

descent, adjusting w_l for each feature $l \in [m]$ as follows.

$$\frac{\partial \mathcal{L}}{\partial w_l} = \frac{\partial}{\partial w_l} \sum_{i=1}^{N} \left(\sum_{j=1}^{m} \mathbf{x}_i(j) \cdot w_j - y_i \right)^2 \tag{1}$$

$$= \sum_{i=1}^{N} 2\mathbf{x}_i(l) \left(\sum_{j=1}^{m} \mathbf{x}_i(j) \cdot w_j - y_i \right) \tag{2}$$

$$= \sum_{i=1}^{N} 2\mathbf{x}_i(l)(\mathbf{x}_i \cdot \mathbf{w} - y_i) \tag{3}$$

The key insight here is that the $\mathbf{x}_i \cdot \mathbf{w} - y_i$ can be computed without view of each individual feature, thus updating w_l only requires $\mathbf{x}_i(l)$. The party that needs to update w_l also owns $\mathbf{x}_i(l)$, so federated gradient descent is possible.

References

[1] Siwei Feng and Han Yu. Multi-participant multi-class vertical federated learning. *CoRR*, abs/2001.11154, 2020.

[2] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *CoRR*, abs/1908.07873, 2019.

[3] Li Wan, Wee Keong Ng, Shuguo Han, and Vincent C. S. Lee. Privacy-preservation for gradient descent methods. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '07, page 775–783, New York, NY, USA, 2007. Association for Computing Machinery.

[4] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. Federated machine learning: Concept and applications. *CoRR*, abs/1902.04885, 2019.